Porting Autoware Nodes to nano-ros

a Safety Island Case Study

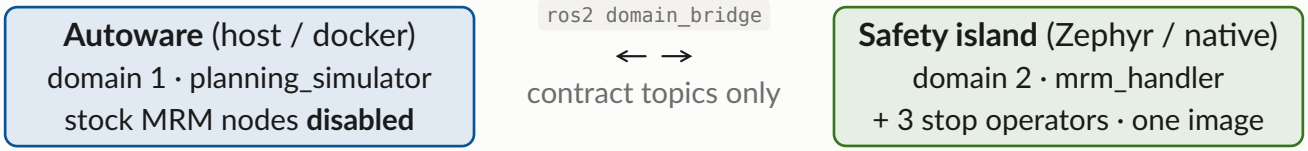
Real Autoware 1.5.0 C++ nodes, near-verbatim, running the MRM chain on an RTOS — co-working with an unmodified Autoware stack.

`github.com/NEWSLabNTU/simple-autoware-safety-island` · nano-ros: `github.com/NEWSLabNTU/nano-ros`

Why a safety island?

When the main compute fails, something small and independent must stop the vehicle.

- Autoware's MRM (Minimum Risk Maneuver) chain normally runs **inside** the same Linux/DDS stack it is supposed to survive.
- Move exactly that chain to a small RTOS image — a **safety island** — and keep the rest of Autoware untouched.
- Two ROS 2 domains, one bridge, one contract:



Failure model: kill the availability heartbeat → the **island** — not Autoware — brings the vehicle to a controlled stop.

What was ported

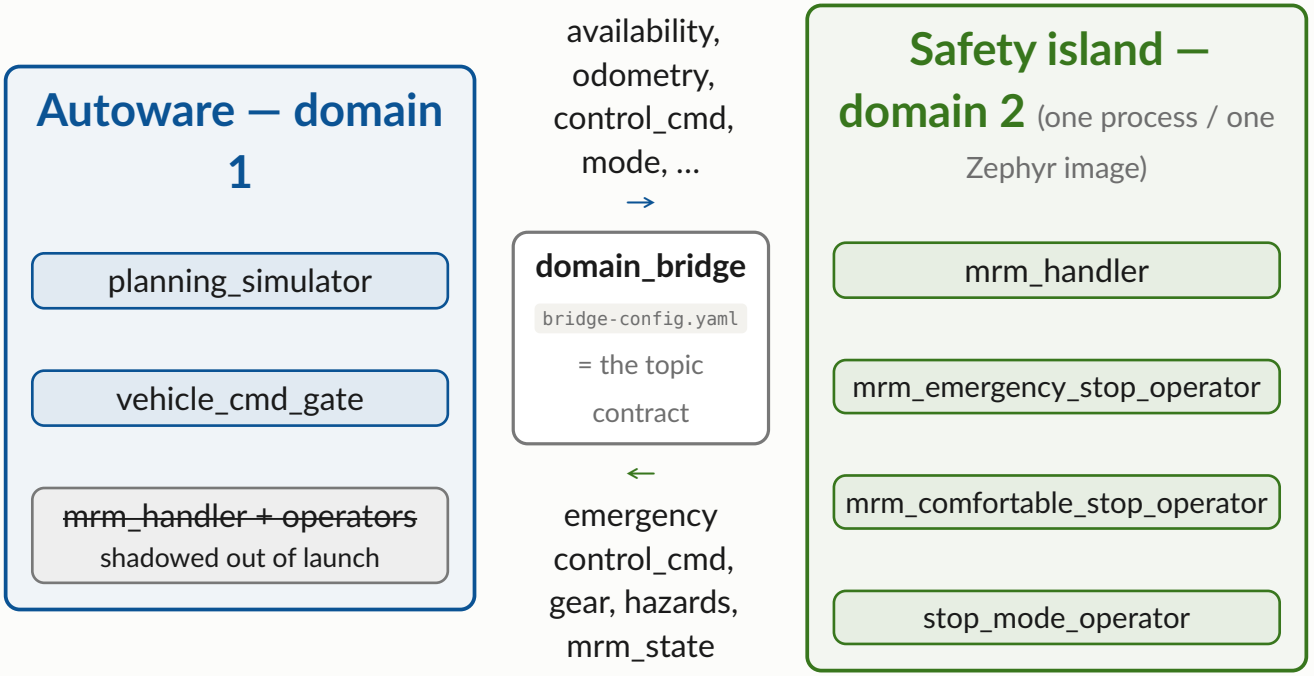
Autoware 1.5.0 · verbatim-first

From `autoware_universe/{system,control}`, easiest → hardest:

Package	LOC	Role
`autoware_mrm_emergency_stop_operator`	~156	jerk-limited hard stop ramp
`autoware_mrm_comfortable_stop_operator`	~131	gentle stop via velocity limit
`autoware_stop_mode_operator`	~110	continuous safe-command emitter
`autoware_mrm_handler`	~600	MRM state machine — the island brain

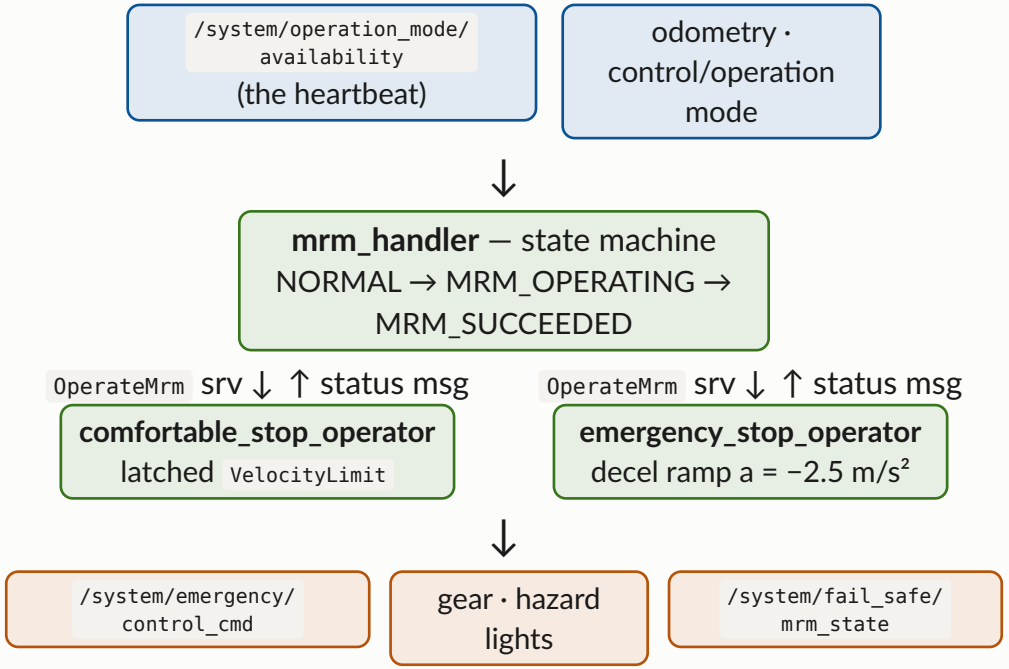
- Ported sources keep the upstream **file layout**, **`package.xml`**, **launch XML**, and an ament-shaped CMake surface — a ROS user recognizes everything.
- Message deps vendored **verbatim**: `autoware_control_msgs`, `autoware_vehicle_msgs`, `autoware_adapi_v1_msgs`, `tier4_system_msgs`, … — stock `rosidl_generate_interfaces()` + `package.xml`, unchanged.

Architecture — two domains, one bridge



Nothing else crosses: `docs/topic-contract.md` is the SSoT, the bridge YAML is the machine artifact. Same island binary runs native (dev loop) and Zephyr.

Inside the island — the MRM chain



Operators tick at 30 Hz (upstream `update_rate`). Handler escalates: comfortable stop → cancel → emergency stop, exactly as in stock Autoware.

Project structure — a 3-role nano-ros workspace

```
CMakeLists.txt              workspace root
src/
  autoware_control_msgs/    ┐ vendored rosidl pkgs
  autoware_vehicle_msgs/    │ (subset of files,
  tier4_system_msgs/  …     ┘  otherwise VERBATIM)
  autoware_mrm_handler/     ┐ ported node pkgs
  autoware_mrm_*_operator/  │ (upstream layout:
  autoware_stop_mode_op…/   ┘  include src launch config)
  island_interfaces/        msg-closure shim (1 FFI union)
  safety_island_bringup/    system.toml + launch XML
                            + params + resolved model
  native_entry/             native entry (fast dev loop)
  zephyr_entry/             Zephyr image entry
demo/                       sim + domain_bridge + scenario
docs/porting-notes.md       the friction log (deliverable!)
```

Three roles:

interface pkgs

verbatim rosidl

node pkgs

your C++, barely touched

bringup + entry

the only nano-ros-specific part

The porting recipe

1 — vendor the message subset

Copy the `.msg` / `.srv` files + `package.xml` + `rosidl_generate_interfaces()` CMake — **unchanged**. nano-ros codegen consumes stock rosidl pkgs.

2 — copy the package

Keep `include/autoware/<pkg>/`, `src/`, `launch/`, `config/*.param.yaml` as-is.

3 — apply six mechanical edits →

4 — wire it up

Add the pkg to workspace CMake + one `[[component]]` block in `system.toml`; `just setup && just build && just run`.

The six edits (per node):

1. Base class: `rclcpp::Node` → `nros::ComponentNode` (rclcpp shape)
2. Ctor: `(const NodeOptions&)` → `(nros::NodeHandle)`
3. Callbacks: `Msg::ConstSharedPtr` → `const Msg&`
4. Register: `RCLCPP_COMPONENTS_REGISTER_NODE` → `NROS_COMPONENT(...)`
5. Topic names: write the **resolved** contract names (remaps not routed yet)
6. Value-init messages: `Response r{};` (generated structs are PODs)

Everything else — the actual node logic — is a diff you can read in one sitting.

Before / after — emergency stop ctor

156 LOC pkg, diff ≈ 20 lines

upstream (rclcpp)

```
MrmEmergencyStopOperator::MrmEmergencyStopOperator(
  const rclcpp::NodeOptions & node_options)
: Node("mrm_emergency_stop_operator", node_options)
{
  params_.update_rate =
    declare_parameter<int>("update_rate");
  sub_control_cmd_ = create_subscription<Control>(
    "~/input/control/control_cmd", 1,
    std::bind(&Op::onControlCommand, this, _1));
  service_operation_ = create_service<OperateMrm>(
    "~/input/mrm/emergency_stop/operate",
    std::bind(&Op::operateEmergencyStop, this, _1, _2));
  pub_status_ = create_publisher<MrmBehaviorStatus>(
    "~/output/mrm/emergency_stop/status", 1);
  timer_ = rclcpp::create_timer(this, get_clock(),
    rclcpp::Rate(params_.update_rate).period(),
    std::bind(&Op::onTimer, this));
}
RCLCPP_COMPONENTS_REGISTER_NODE(
  autoware::mrm_emergency_stop_operator::
    MrmEmergencyStopOperator)
```

ported (nano-ros)

```
MrmEmergencyStopOperator::MrmEmergencyStopOperator(
  ::nros::NodeHandle handle)
: ::nros::ComponentNode(handle,
    "mrm_emergency_stop_operator")
{
  params_.update_rate =
    declare_parameter<int64_t>("update_rate", 30);
  NROS_SUBSCRIBE(Control, onControlCommand,
    "/control/command/control_cmd");
  ::nros::bind_service<OperateMrm, Op,
      &Op::operateEmergencyStop>(
    node(), "/system/mrm/emergency_stop/operate", this);
  pub_status_ = create_publisher<MrmBehaviorStatus>(
    "/system/mrm/emergency_stop/status");
  NROS_CREATE_TIMER(
    1000 / params_.update_rate, onTimer);
}
NROS_COMPONENT(
  autoware_mrm_emergency_stop_operator::
    MrmEmergencyStopOperator);
```

Timer/pub/sub bodies, the ramp math, the state handling — **byte-identical** to upstream.

CMake and launch — ament-shaped on purpose

Node pkg CMake — `ament_cmake_auto` becomes one verb:

```
find_package(nano_ros REQUIRED)
find_package(tier4_system_msgs REQUIRED)

nano_ros_add_node(mrm_emergency_stop_operator
  CLASS  autoware_mrm_emergency_stop_operator::
         MrmEmergencyStopOperator
  HEADER autoware/mrm_emergency_stop_operator/
         mrm_emergency_stop_operator_core.hpp
  SHAPE  rclcpp
  SOURCES src/.../mrm_emergency_stop_operator_core.cpp)
```

No `main()` — the node pkg builds a component; the **entry** pkg owns boot.

Bringup — launch XML stays ROS 2 schema, verbatim:

```
<launch>
  <node pkg="autoware_mrm_handler"
        exec="mrm_handler" name="mrm_handler"/>
  <node pkg="autoware_mrm_emergency_stop_operator"
        exec="mrm_emergency_stop_operator" .../>
  ...
</launch>
```

`system.toml` adds what ROS has no word for — domain, RMW, components, tiers:

```
play_launch resolve safety_island.launch.xml
  --system system.toml -o system_model.yaml
```

One resolved model bakes into **both** the native and the Zephyr entry.

Under the hood — the real commands `just` is only the front door

Everything below is plain CLI — trivially portable to your own Makefile / CI scripts.

0 — environment (once per shell)

```
source $NANO_ROS_ROOT/activate.sh   # nros CLI + play_launch
```

1 — resolve the system model (`just setup`)

```
play_launch resolve \
  src/safety_island_bringup/launch/safety_island.launch.xml \
  --system src/safety_island_bringup/system.toml \
  -o src/safety_island_bringup/config/system_model.yaml
```

2 — native build (`just build` ; plain CMake)

```
NROS_EXECUTOR_MAX_CBS=32 \
  cmake -S . -B build -DNANO_ROS_ROOT=$NANO_ROS_ROOT
cmake --build build -j
```

3 — run the island (`just run`)

```
ROS_DOMAIN_ID=2 \
LD_LIBRARY_PATH=~/.nros/sdk/cyclonedds/0.10.5-nros1/lib \
  ./build/src/native_entry/native_entry
```

4 — Zephyr build + run (`just zephyr-build/-run` ; plain west)

```
source $NANO_ROS_ROOT/zephyr-workspace/env.sh
NROS_EXECUTOR_MAX_CBS=32 NROS_INTERFACE_SEARCH_PATH=$PWD/src \
  west build -b native_sim/native/64 -d build-zephyr \
    src/zephyr_entry -- \
    -DCONF_FILE="prj.conf;prj-cyclonedds.conf" \
    -DCMAKE_PREFIX_PATH=$NANO_ROS_ROOT
./build-zephyr/zephyr/zephyr.exe
```

Gotchas the justfile encodes: pin `LD_LIBRARY_PATH` to the SDK cyclonedds (a sourced ROS env shadows it → SIGSEGV); clean-rebuild after changing the compile-time knobs.

What to expect — API surface

porting-notes 01–05, 13

#	Upstream expects	What you do	Status
01	`create_service`, params, `now()` on `rclcpp::Node` — compat header covers pub/sub/timer/QoS/log only	derive `nros::ComponentNode` (rclcpp shape); services via `nros::bind_service`	open
04	`Msg::ConstSharedPtr` callbacks	`const Msg&` — no shared_ptr message ownership	by design
05	`rclcpp::Clock` / `Time` arithmetic	`nros_cpp_time_ns()` + dt from message stamps (monotonic epoch)	open
13	C++17 (`std::optional`)	C++14 surface — sentinel flag + value; mechanical	by design

All of these are **local, mechanical** edits — the compiler finds every site for you.

What to expect — semantics

porting-notes 06, 07, 09, 14

#	Upstream expects	What you do	Status
07	`~/private` names + launch `<remap>` routing	write resolved contract names in-source; launch XML keeps remaps as documentation	open
06	params injected from `*.param.yaml` at launch	param-file values become in-code `declare_parameter` defaults (node-local)	open
09	rosidl C++ zero-initializes messages	**value-init**: `Response r{};` — generated structs are PODs (garbage leaked on the wire otherwise)	open — fix planned
14	`polling_subscriber`, blocking service futures	cache-latest member subs; send-and-poll service clients (no nested spin in a timer cb)	open

#09 is the one that **bites silently** — make `{}` a porting reflex.

What to expect — build & scale

5 nano-ros fixes landed same-day

#	Friction → outcome	Status
08	two interface pkgs on one link line → duplicate FFI symbols → nano-ros now builds one topo-last superset crate; also: msg constants as struct members (`MrmBehaviorStatus::AVAILABLE`)	fixed in nano-ros
10	`rclcpp::QoS(5)` / `.transient_local()` spelling → depth ctor added; ported code unchanged	fixed in nano-ros
16	type-descriptor registry cap 64, overflow **silent** (island registers ~86 types) → cap 256 + override knob	fixed in nano-ros
18	Zephyr minimal libcpp: stub `<new>` broke every component factory → placement-new forms declared	fixed in nano-ros
11	executor callback slots = compile-time env knob (`NROS_EXECUTOR_MAX_CBS`); boot fails loud (`Full`) → export 16+; codegen-derived sizing filed	open
12/15	one msg-closure per workspace (`island_interfaces` shim); full-pkg deps drag srv IDL embedded idlc rejects → vendor msg-only subsets	open

The protocol: every friction point → `porting-notes.md` entry → nano-ros issue → fix **upstream**, not a local fork.

Zephyr: one image, four nodes

Entry pkg = prj.conf + one CMake verb:

```
CONFIG_NROS=y
CONFIG_NROS_CPP_API=y
CONFIG_STD_CPP14=y
CONFIG_NETWORKING=y  CONFIG_NET_UDP=y ...
# sized up for 30+ DDS entities:
CONFIG_MAX_PTHREAD_MUTEX_COUNT=1024
```

```
nano_ros_add_executable(zephyr_entry
  BOARD zephyr  TYPED  DEPLOY zephyr
  MODEL .../config/system_model.yaml)
```

Same model as native — no `[deploy]` pins, so nodes stay board-agnostic.

Build & run:

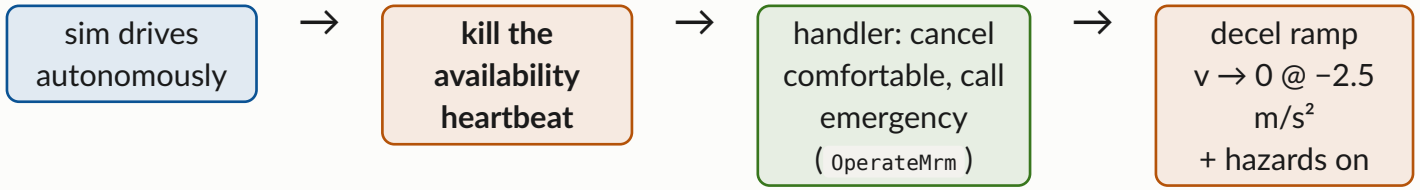
```
just zephyr-build   # west build -b native_sim/native/64
just zephyr-run     # ./build-zephyr/zephyr/zephyr.exe
```

Talking to it from the host: native_sim bakes multicast-off, unicast-peer discovery — the host must mirror it:

```
just host-env   # prints CYCLONEDDS_URI:
# lo iface, AllowMulticast=false,
# Peer 127.0.0.1, ParticipantIndex auto
```

With that: the **full e2e passes against `zephyr.exe` — identical result to native**.

The demo — heartbeat loss, island stops the car



- Verified end-to-end on **native** (P3) and on the **Zephyr image** (P4) — same scenario, same transitions, `/system/fail_safe/mrm_state` → `MRM_OPERATING`.
- Co-sim (P5): host Autoware 1.5.0 `planning_simulator` + `domain_bridge`; stock MRM nodes shadowed out of the launch — the island is the **only** MRM path.

```
just demo-sim          # planning_simulator, domain 1
just demo-bridge       # domain_bridge with the contract yaml
just run               # the island, domain 2
just demo-kill-heartbeat
```

Scorecard

Worked well

- Vendored rosidl pkgs build **verbatim** — they even double as the host colcon overlay
- Node logic ports near-verbatim; ~600-LOC state machine unchanged
- Launch XML consumed as-is; ament-shaped CMake
- QoS chains (`transient_local` latched pubs) just work
- Same sources, native ↔ Zephyr, identical e2e behavior
- Friction→issue→same-day-fix loop with nano-ros mainline

Gaps (rclcpp-compat trajectory)

services / params / clock in the compat header	next
`<remap>` routing + `~/` expansion	planned
param-file projection at launch	planned
zero-initialized generated messages	planned
service clients with futures, callback groups	open
executor sizing derived from the model	filed

Every gap has a documented, mechanical workaround — see `docs/porting-notes.md` (19 entries).

Takeaways — you can port your own nodes

1. **The port is mostly a copy.** Msg pkgs verbatim; node pkgs keep their layout; six mechanical edits per node, and the compiler finds every site.
2. **Your ROS 2 knowledge transfers.** `package.xml`, launch XML, QoS spellings, the ament CMake shape — all recognizable; `system.toml` is the only new file.
3. **The friction is finite and logged.** 19 notes from four packages; 5 fixes already in nano-ros mainline — the list **shrinks** as compat grows.
4. **The payoff is real**: the actual Autoware MRM chain on an RTOS image, stopping the vehicle when the main stack dies.

Start here: `github.com/NEWSLabNTU/simple-autoware-safety-island` — README quick start, `docs/porting-notes.md`, `docs/topic-contract.md`

nano-ros: `github.com/NEWSLabNTU/nano-ros` — RFC-0044 (rclcpp shape), RFC-0028 (safety / E2E), the `nros` CLI + `play_launch`